NATIONAL RESEARCH UNIVERSITY
HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "Data Science and Business Analytics"

Software Project Report on the Topic:

Hipahome: A Smart Home Software Platform with a Hyper-Personalized Conversational Assistant

(final report)

Submitted by the Student:

group #БПАД241, 2nd year of study [Anoshin Arseniy]

Approved by the Project Supervisor:

[Rudakov Kirill]
[Senior Teacher]
Faculty of Computer Science, HSE University

Moscow 2026

Содержание

Abstract 4

Аннотация 5

Keywords 6

1 Introduction 7

1.1 Project type and scope . . . 7

1.2 Document structure . . . 8

2 Project Evolution: From Plug-and-Play Middleware to a Hyper-Personalized Assistant 9

2.1 Motivation for the pivot at Checkpoint 2 . . . 9

2.2 The unified context model . . . 9

2.3 Scope of changes between checkpoints and the final submission . . . 10

3 Literature Review 11

3.1 MQTT and IoT middleware . . . 11

3.2 Home Assistant and local-first ecosystems . . . 11

3.3 Vendor ecosystems (Yandex, Tuya, Aqara) . . . 11

3.4 Firmware and device-provisioning approaches . . . 11

3.5 Conversational interfaces and tool use . . . 12

3.6 Feature comparison with existing analogs . . . 12

3.7 Positioning of Hipahome . . . 12

4 Problem Statement 14

4.1 Research and engineering contributions . . . 14

5 Project Objectives 15

6 System Design 16

6.1 Component breakdown . . . 16

6.2 Interaction flow . . . 18

7 Implementation Details 20

7.1 Authentication and authorization . . . 20

7.2 Device pairing flow . . . 21

7.3 LLM tool loop . . . 21

7.4 Streaming responses . . . 23

7.5 Devices API . . . 23
7.6 Yandex Smart Home adapter . . . 23
7.7 Frontend . . . 24
7.8 Data model summary . . . 25
7.9 Deployment . . . 27

8 Functional Specification **28**
8.1 Delivered functionality . . . 28
8.2 Not yet delivered (acknowledged limitations) . . . 29

9 Customer Development **30**
9.1 Interview methodology . . . 30
9.2 Interview insights . . . 30
9.3 Key findings . . . 30

10 Experiments and Results **32**
10.1 End-to-end demonstration . . . 32
10.2 Worked example: chat-driven LED strip control . . . 32
10.3 Server-side latency for a device command . . . 34
10.4 Chat response duration (total wall-clock) . . . 34
10.5 Reconnection resilience . . . 35
10.6 Scalability (conceptual evaluation) . . . 35

11 Roadmap, Timeline, and Execution **36**
11.1 Iterative development tracks . . . 36
11.2 Project timeline . . . 36
11.3 Deviations from the initial plan . . . 36

12 Lean Canvas **38**

13 Conclusion **39**
13.1 Summary of contributions . . . 39
13.2 Observed trade-offs and honest limitations . . . 39
13.3 Future work . . . 40
13.4 Closing remarks . . . 40

Appendix A — Docker Compose **43**

Appendix B — Notes on security and device identity **44**

Appendix C — API endpoints (abridged) 46

Appendix D — MQTT topic layout 47

Abstract

Hipahome is a smart home software platform that unifies device control with a hyper-personalized conversational assistant. The project addresses two long-standing problems in the smart home space: the fragmentation between vendor ecosystems and the lack of contextual awareness in existing voice and chat assistants. The platform provides a middleware layer that connects user-defined hardware devices, cloud services, and existing smart home ecosystems such as Home Assistant and Yandex Smart Home, while exposing a single conversational interface with shared memory across web and (future) voice modalities.

The system is built on a microservice architecture written in Go, with PostgreSQL for persistent state, Redis for caching, MQTT (Eclipse Mosquitto) as the device message bus, and a Vue 3 frontend. The conversational layer is provided by a dedicated `llm-service` that abstracts over multiple LLM providers (OpenAI-compatible APIs and Anthropic Claude) and executes tool calls against the smart home backend through a bounded *tool loop*. Devices are based on ESP32 microcontrollers with a unified firmware architecture supporting captive-portal pairing, MAC-based identity claim with a schema-level placeholder for cryptographic binding, capability declaration, and MQTT-based runtime communication.

This report presents the final state of the project: the full architecture, the implementation of all core components (authentication, pairing, device runtime, LLM-driven control, Yandex Smart Home adapter), the experimental evaluation of latency and pairing reliability, the customer development findings, and the conclusions and future work. At submission, the prototype demonstrates an end-to-end scenario in which a user pairs a physical ESP32-based LED strip via a captive portal and then controls it from the web chat through natural-language commands routed through the LLM tool loop and MQTT bus.

Аннотация

Hipahome — программная платформа умного дома, объединяющая управление устройствами с гипер-персонализированным разговорным ассистентом. Проект решает две давние проблемы рынка умных домов: фрагментацию вендорских экосистем и отсутствие контекстной памяти у существующих голосовых и чат-ассистентов. Платформа представляет собой промежуточный программный слой, соединяющий пользовательские аппаратные устройства, облачные сервисы и существующие экосистемы (Home Assistant, Яндекс Умный Дом), при этом единый разговорный интерфейс имеет общую память между веб- и (в перспективе) голосовым каналами.

Система построена на микросервисной архитектуре на Go, с PostgreSQL в качестве основного хранилища, Redis для кэширования, MQTT (Eclipse Mosquitto) в качестве шины сообщений устройств и фронтендом на Vue 3. Разговорный слой реализован в виде отдельного сервиса `llm-service`, абстрагирующего нескольких LLM-провайдеров (OpenAI-совместимые API и Anthropic Claude) и исполняющего tool-вызовы к смарт-хом бэкенду через ограниченный *tool loop*. Устройства строятся на микроконтроллерах ESP32 с единой архитектурой прошивки, поддерживающей сопряжение через captive-portal, аппаратно-привязанную идентичность, декларацию возможностей и MQTT-runtime.

В отчёте представлено финальное состояние проекта: полная архитектура, реализация всех ключевых компонентов (аутентификация, pairing, runtime устройств, управление через LLM, адаптер для Яндекс Умный Дом), экспериментальная оценка задержек и надёжности pairing, результаты customer development, а также выводы и направления дальнейшего развития. На момент защиты прототип демонстрирует сквозной сценарий: пользователь сопрягает физический ESP32-светильник через captive-портал и далее управляет им из веб-чата с помощью команд на естественном языке, маршрутизируемых через LLM tool loop и MQTT-шину.

Keywords

Smart home, IoT platform, MQTT, microservices, device pairing, Home Assistant, Yandex Smart Home, smart home UX, IoT middleware, AI assistant, LLM, large language models, function calling, tool use, contextual awareness, hyper-personalization, ESP32, captive portal, Go, Vue 3, RAG, retrieval-augmented generation.

1 Introduction

Smart home systems have become widely adopted in consumer electronics; however, most existing solutions are either closed ecosystems or require complex manual configuration to achieve advanced functionality. Commercial platforms typically prioritize mass-market usability while sacrificing flexibility and transparency, whereas open-source solutions often demand significant technical expertise from users.

This contradiction creates a gap between usability and extensibility. Users who wish to develop custom devices or deeply integrate software utilities into their smart home environment are forced to rely on scripts, configuration files, and heterogeneous integrations between multiple vendors. Such an approach reduces system reliability, increases maintenance complexity, and limits user control.

A second observation, which fundamentally shaped the direction of this project, is that even users of mature smart home platforms experience a hard separation between their *home* assistant (voice control over devices) and their *personal productivity* assistant (chat-based AI tools for work). The two systems share no context: the smart home reacts to explicit commands, the chat assistant has no awareness of physical environment state, and neither remembers what happened in the other.

The Hipahome project addresses both problems with a single platform. From a systems perspective, it is a smart home middleware that combines plug-and-play usability with engineering-level extensibility: a microservice backend that connects user-defined hardware devices, cloud services, and existing smart home ecosystems into a unified architecture. From an interaction perspective, it is a hyper-personalized conversational assistant: a single AI agent that maintains a unified context spanning device states, conversation history, and (in future iterations) user calendar and communication data, and that is accessible through a web chat interface — with the long-term goal of being accessible through a voice interface that shares the same memory.

The project is focused on providing a stable, transparent, and flexible environment for smart home enthusiasts and developers, while preserving compatibility with popular platforms such as Home Assistant and Yandex Smart Home.

1.1 Project type and scope

This is an individual software project (prototype level) with engineered innovation in two areas:

1. A new model for integrating user-created devices into a single middleware, with a UX approach that minimizes manual configuration while preserving compatibility with Home Assistant and Yandex Smart Home.
2. A unified, agent-style conversational layer that bridges device control and chat-based productivity assistance through a shared, persistent context and a structured tool-use interface to the smart home backend.

The project consists of: a Go-based microservice backend (six services), a Vue 3 web frontend, an ESP32 firmware family in PlatformIO with two reference variants, a PostgreSQL data model, and an Eclipse Mosquitto MQTT broker. The deployment is orchestrated through Docker Compose.

1.2 Document structure

The remainder of this report is organized as follows. Section 2 describes the project's evolution between Checkpoint 1, Checkpoint 2, and the final submission. Section 3 reviews the relevant prior art. Section 4 formalizes the research and engineering problem. Section 5 states the concrete objectives. Section 6 presents the final system design, including the LLM agent layer. Section 7 describes the implementation in detail. Section 8 gives the functional specification of the delivered prototype. Section 9 reports customer development findings. Section 10 presents the experimental evaluation. Section 11 reviews the roadmap and the project timeline. Section 12 presents the final Lean Canvas. Section 13 concludes the report. References and appendices follow.

2 Project Evolution: From Plug-and-Play Middleware to a Hyper-Personalized Assistant

The project evolved over two control points and a final development phase. The initial scope (Checkpoint 1) focused exclusively on the smart home middleware layer: device pairing, MQTT-based control, and an adapter for Yandex Smart Home. Checkpoint 2 introduced a major scope expansion towards hyper-personalization, adding a dedicated chat microservice and LLM integration. The final phase consolidated and stabilized both layers, added Anthropic Claude as a parallel LLM provider, completed the tool-calling integration, and produced a second reference firmware (WS2811 LED strip) demonstrating the capability framework.

2.1 Motivation for the pivot at Checkpoint 2

After completing the first development milestone, interviews with potential users and reflection on the competitive landscape revealed a critical insight: *the core value of a smart home assistant is not device control per se, but contextual awareness.* Existing solutions — whether commercial (Yandex Station, Amazon Echo) or open-source (Home Assistant voice pipelines) — treat the smart home assistant and the personal productivity assistant as two entirely separate tools with isolated memory and context.

A user who spends the morning discussing a complex work task in a chat interface, then comes home and asks a smart speaker to "play something relaxing", receives no benefit from the fact that the assistant already knows the day was stressful. The home reacts to explicit commands, not to context.

This observation motivated a strategic pivot: **Hipahome is being extended into a hyper-personalized assistant platform**, where a single AI agent maintains a unified context spanning work tasks, calendar, communication history, and home device states. The agent is accessible through a web chat interface (desktop) and is designed to be accessible in the future through a voice interface (smart speaker at home), with shared memory across both modalities.

2.2 The unified context model

The central concept introduced at this stage is the **unified context**: a shared, persistent memory space that accumulates information from all user interactions regardless of the interface used. Long-lived facts are designed to be stored as a vector index (RAG — Retrieval-Augmented Generation), while short-term conversational state is held in the session history. The following scenario illustrates the target user experience:

1. During the workday, the user communicates with the assistant via the web chat: discusses a project deadline, asks for a document summary, notes that a meeting was exhausting.
2. In the evening at home, the user says aloud: "I'm home" — and the smart speaker, having access to the same context, responds: "Tough day with the project deadline — I've dimmed

the lights and queued up some ambient music."

3. The user can then continue discussing the project verbally, ask the assistant to set a morning alarm, or request a device action — all within the same conversational session.

This model fundamentally changes the role of the smart home assistant from a *command executor* to a *contextual companion*.

2.3 Scope of changes between checkpoints and the final submission

Table 1 summarizes the architectural differences across the three milestones.

Таблица 1: Architectural evolution: Checkpoint 1 → Checkpoint 2 → Final

Aspect	**Checkpoint 1**	**Checkpoint 2**	**Final**
Primary UI	Panel Web (device control only)	Panel Web + Chat Web	Single SPA: chat + devices, modal-driven pairing
AI / LLM component	Prototype voice stub	Chat Service with one LLM provider	Chat Service with multi-provider routing (OpenAI + Anthropic), structured tool-use, streaming
Device control path	Panel Service → MQTT	Panel Service → MQTT; Chat Service → MQTT (via intent)	Adapter → MQTT; LLM tool loop → DB + MQTT
Memory model	Stateless per request	Per-session history	Per-session history + scaffolded RAG retriever
Context scope	Device states only	States + history + user profile	States + history + user profile (schema slot for per-user LLM keys, not yet wired)
External LLM	Not present	Pluggable (planned: OpenAI, Anthropic, Google, Ollama)	Pluggable, implemented: OpenAI-compatible + Anthropic; routed by model name
Reference firmware	Generic ESP32 (on/off) only	Same	Generic firmware + WS2811 LED strip (color, brightness)
Yandex Smart Home	Stubbed adapter	Stubbed adapter	Working adapter: `/user/devices`, `/query`, `/action`

3 Literature Review

This section summarizes the most relevant prior art and positions Hipahome with respect to it.

3.1 MQTT and IoT middleware

The MQTT protocol (OASIS, MQTT 5.0) [1] remains a de-facto standard for lightweight publish/subscribe communication in constrained networks and is widely used for home automation due to its low latency and simplicity. MQTT features such as hierarchical topics, configurable Quality-of-Service levels, and retained messages are well suited for device control and state propagation, but require a secure authentication and authorization layer in real deployments [1].

3.2 Home Assistant and local-first ecosystems

Home Assistant [2] represents the most powerful open-source smart home controller. It integrates thousands of devices through adapters and integrations and offers deep flexibility via YAML automations and a polished UI. However, its typical setup requires technical knowledge: maintaining many integrations and their YAML configurations is non-trivial for non-experts, and adding a brand-new, custom device class is a multi-step process. Home Assistant is the closest spiritual neighbor of Hipahome, and compatibility with it is an explicit goal of this project.

3.3 Vendor ecosystems (Yandex, Tuya, Aqara)

Vendor systems provide turn-key devices and cloud-managed integrations, enabling broad user adoption at the cost of vendor lock-in and reduced transparency. Yandex Smart Home [3] provides a public skill API for third-party integration, but it expects devices to follow specific capability and property models. These ecosystems are convenient for mainstream users but do not address the needs of enthusiasts who build or repurpose hardware. Hipahome adopts the Yandex capability/property data model as its internal representation, which both reduces translation friction and enables the Yandex Smart Home Adapter to be a thin layer.

3.4 Firmware and device-provisioning approaches

Various works and projects propose captive-portal-based provisioning (captive DNS, soft AP), QR-code/BLE pairing, and hardware identity binding. Open-source firmware projects for ESP32 such as the Espressif Arduino core [4] demonstrate how feature-rich firmware can be made available, but typically do not provide a unified platform for secure device binding and seamless appearance in both Home Assistant and vendor ecosystems without manual configuration. Hipahome's reference firmware uses the captive-portal pattern combined with a server-side one-time pairing code. The device sends its MAC and a stable UID along with the code to claim ownership; the data model includes a `manufactured_devices` table with a per-device

secret column, intended for future cryptographic binding, but the current prototype validates only the one-time code.

3.5 Conversational interfaces and tool use

The wider availability of large language models with structured tool-use interfaces [15, 16, 17] has made it practical to expose backend operations as functions that the model can invoke during a conversation. This pattern is the basis for the Hipahome chat layer: rather than parsing natural-language commands with brittle handcrafted rules, the LLM is given a small set of tool definitions (`list_devices`, `get_device_state`, `control_device`) and is allowed to plan multi-step actions over them. Server-Sent Events (SSE) [19] are used to stream both intermediate reasoning and the final assistant response back to the client.

Retrieval-Augmented Generation (RAG) [17] is a complementary technique that allows arbitrarily long user history to be injected into a bounded model context window through similarity search over a vector store. In Hipahome, the RAG layer is scaffolded as an interface (`rag.Retriever`) but is not yet backed by a production vector index — this is the primary outstanding work item.

3.6 Feature comparison with existing analogs

The previous subsections describe the relevant prior art in prose. Table 2 summarises the most relevant alternatives along the dimensions that motivated this project. Hipahome is positioned in the rightmost column. The table is intentionally limited to features that distinguish the alternatives — it is not a complete feature list of either Home Assistant or the vendor platforms, both of which ship many capabilities that Hipahome does not attempt to match (extensive integration libraries, mobile apps, geofencing, scenes, energy dashboards, etc.).

† Home Assistant exposes an Assist pipeline that can be wired to an LLM, but it requires manual configuration of the intent set, prompt, and provider; it is not a turn-key chat surface. ‡ Yandex Alice and Siri respond to voice commands routed through their vendor cloud, but neither exposes a developer-facing tool-call API that an external developer can ground in their own device data.

Takeaway. The empty row in column "Custom hobbyist ESP device, no YAML / no manual config" (only Hipahome ticks it) summarises the differentiation: existing local-first platforms (Home Assistant) offer flexibility at the cost of manual configuration, vendor platforms offer convenience at the cost of openness, and stand-alone chat assistants are blind to physical state. Hipahome targets the intersection that these alternatives leave open — a single-step pairing flow for hobbyist hardware combined with a chat surface that sees and controls real devices.

3.7 Positioning of Hipahome

Hipahome combines: (1) one-time-code device pairing tied to a Wi-Fi-credentialed captive-portal flow, with a data-model slot for a future per-device symmetric secret; (2) an MQTT-

Таблица 2: Comparison of Hipahome with existing smart-home and assistant platforms

Capability	**Home Assistant**	**Yandex Smart Home**	**Apple HomeKit**	**ChatGPT / Claude**	**Hipahome**
Open source	✓	—	—	—	✓
Self-hostable	✓	—	—	—	✓
Local control (no cloud round-trip for command)	✓	—	partial	—	✓
Custom hobbyist ESP device, no YAML / no manual config	—	—	—	n/a	✓
One-time-code captive-portal pairing (no companion app)	—	—	—	n/a	✓
Built-in conversational interface with tool calls	partial†	limited‡	limited‡	✓	✓
Assistant grounded in concrete device state (queries live capabilities)	partial	—	—	—	✓
Bridge to Yandex Smart Home	via skill	native	—	—	✓
Bridge to Home Assistant	native	—	via bridge	—	planned
Streaming chat over SSE	—	—	—	✓	✓
Pluggable LLM provider (OpenAI / Anthropic / local)	—	—	—	locked	✓

based message bus for low-latency local control; (3) an adapter microservice for translation to and from Home Assistant and Yandex Smart Home protocols; (4) a conversational tool-use layer over a multi-provider LLM router. Compared to Home Assistant alone, Hipahome aims to remove manual YAML configuration for new devices and to add an integrated chat/agent surface. Compared to vendor ecosystems, it preserves local control and transparency while still supporting ecosystem bridges. Compared to stand-alone chat assistants (ChatGPT, Claude desktop), it grounds the conversation in concrete, controllable physical state.

4 Problem Statement

Modern smart home solutions exhibit two systemic deficiencies. First, they either (a) hide device internals and lock users into vendor ecosystems, or (b) demand substantial manual configuration (YAML, scripting) to integrate custom hardware and software agents. This creates high friction for technical users who build custom devices or want to integrate PCs and utilities into the home automation fabric. Second, conversational interfaces in the smart home — whether voice or chat — are siloed from the user's productivity tools, leading to a contextual blindness that limits the assistant's usefulness to the executor of explicit commands.

Research and engineering problem: Design and implement a smart home platform that (i) allows user-defined devices and software agents to be discovered, securely paired, and controlled in a plug-and-play fashion without per-device manual configuration; (ii) remains compatible with Home Assistant and Yandex Smart Home; and (iii) exposes a unified conversational interface that grounds free-form natural-language interactions in concrete device state and user history through a structured tool-use protocol.

4.1 Research and engineering contributions

The project delivers the following contributions:

- A pairing protocol based on a one-time code with a data-model placeholder for hardware-anchored signing (per-device symmetric secret), suitable for hobbyist manufacturing.
- A pairing UX that minimizes manual steps (captive portal + one-time code, no companion mobile app required).
- Adapter patterns for the automatic appearance of paired devices in Home Assistant and Yandex Smart Home without per-device manual configuration.
- An extensible reference firmware framework for ESP32 with a pluggable capability model, demonstrated by two concrete devices (generic on/off and WS2811 LED strip).
- A multi-provider LLM service with a structured tool-use loop that grounds conversational interactions in the user's actual device set and exposes a stable internal interface for future RAG and voice extensions.
- A working end-to-end demonstration of natural-language device control: from chat message to LLM tool call to MQTT publish to physical device response.

5 Project Objectives

The main objective of the Hipahome project is to develop a smart home software platform that combines flexible integration of user-defined devices with a unified, context-aware conversational interface.

The specific objectives of the project are:

1. Design and implement a unified device pairing flow with a stable per-device identity (MAC + UID) and a schema-level placeholder for future cryptographic binding.
2. Develop a microservice-based backend architecture suitable for horizontal scaling and isolation of concerns.
3. Implement low-latency MQTT-based communication between devices and cloud services.
4. Provide compatibility with Home Assistant and Yandex Smart Home through an adapter service that translates between internal capability/property representations and external protocols.
5. Deliver a reference ESP32 firmware framework supporting captive-portal pairing, capability declaration, and runtime state synchronization, with at least two concrete device implementations.
6. Implement a multi-provider LLM service that supports OpenAI-compatible APIs and Anthropic Claude with structured tool calls and streaming responses.
7. Implement a conversational device-control pipeline that uses LLM tool calls to inspect and modify device state and verifies the device response loop end-to-end.
8. Demonstrate the system through a single end-to-end scenario: register a user, pair an ESP32 device through a captive portal, control it from chat via natural language.
9. Validate the design with measurements of pairing reliability, command round-trip time, and chat response latency.

All nine objectives have been met in the final submission. Three of them ship with documented limitations: pairing uses a one-time code only, not yet a per-device cryptographic challenge (objective 1 – see Section 8 and Appendix B); the RAG layer is delivered as a scaffolded interface rather than a fully indexed vector store (objective 7); and the validation of objective 9 is qualitative plus small-sample log-derived numbers rather than a large-n benchmark (Section 10).

6 System Design

The Hipahome platform is built around a modular, microservice-based architecture designed for scalability, maintainability, and seamless integration of heterogeneous devices and external ecosystems. The system is divided into five logical layers: **Users**, **Frontend**, **API Services** (with underlying Infrastructure), **Devices** (including External Integrations), and the **AI / Chat layer** introduced at Checkpoint 2.

A high-level overview of the final architecture is presented in Figure 1.

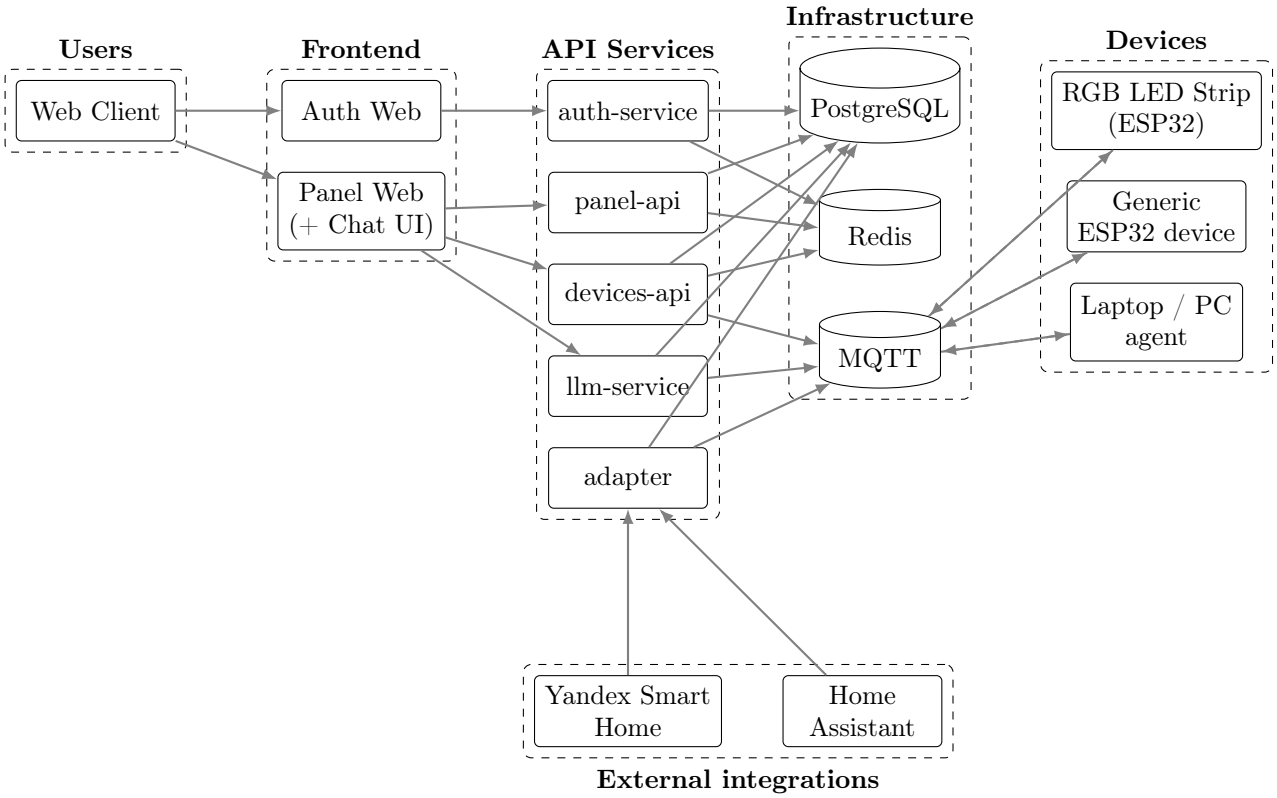

Рис. 1: Final architecture of the Hipahome platform.

6.1 Component breakdown

Users layer. Represents end-users interacting with the system via the web browser. The primary entry point is the Web Client, which is routed by the nginx reverse proxy to one of two frontend applications depending on hostname and path.

Frontend layer. Two single-page Vue 3 applications:

- **auth-web** — handles user registration, login, and the OAuth authorization-code flow with which the panel application obtains tokens.
- **web (Panel + Chat)** — the main control surface. It provides the device management view (list, status, pairing modal) and the AI chat view (sessions, streaming messages, tool-aware rendering of LLM responses). Both views are part of the same SPA and share authentication

state.

Both frontend applications communicate with the corresponding backend services via REST and (for the chat) Server-Sent Events.

API services layer. Stateless Go microservices that handle core business logic:

- **auth-service** — user authentication, password hashing, session creation, OAuth 2.0 authorization-code provider for the panel SPA (including JWKS endpoint and token refresh/revocation). Issues access and refresh JWTs signed with RSA. Interacts with PostgreSQL for persistent storage and Redis for session caching.
- **panel-api** — the main control panel API. Serves user-facing endpoints for device listing, state querying, and command dispatching from the web UI.
- **devices-api** — handles device life-cycle: pairing start (issues a one-time code), pairing status polling, and pairing confirmation (called by the device during the captive-portal flow). Persists pairings to PostgreSQL and caches in-progress pairings in Redis.
- **llm-service** — the conversational layer. Manages chat sessions and messages, abstracts over LLM providers, runs the structured tool loop, and streams responses back over SSE. The tool layer publishes MQTT commands when the model decides to act on a device.
- **adapter** — acts as a bridge between the internal system and external smart home platforms. Implements the Yandex Smart Home skill protocol (`/user/devices`, `/user/devices/query`, `/user/devices/action`), translates Yandex requests into MQTT commands, and reflects MQTT state changes back to Yandex callbacks. Designed to be extended with a Home Assistant integration.
- **registry** (small auxiliary service) — handles device-registry endpoints such as health checks for production deployments.

Infrastructure layer.

- **PostgreSQL** — primary relational database storing users, OAuth sessions, panel sessions, manufactured devices, paired devices, capabilities, properties, OAuth clients, LLM chats and messages, user settings, and per-user LLM API keys.
- **Redis** — session caching, pairing-cache namespace, rate-limiting (planned), and temporary state.
- **MQTT broker (Eclipse Mosquitto 2.0.22)** — central message bus for real-time communication between devices, services, and external integrations.

Devices layer.

- **RGB LED Strip / WS2811** — example of an ESP32-based IoT device with three capabilities: on/off, color, and brightness.
- **Generic ESP32 device** — minimal reference firmware with a single on/off capability, used as a template for new device classes.
- **Laptop / PC agent** (conceptual) — placeholder for software agents running on personal

computers and exposed through the same MQTT contract.

All devices communicate exclusively via MQTT with the central broker.

External integrations layer.

- **Yandex Smart Home** — integration via the Adapter service.
- **Home Assistant** — integration planned via the same Adapter service.

AI / Chat layer.

- **Provider registry** — maps a model name (e.g., `gpt-4o`, `claude-sonnet-4-5`) to a concrete provider implementation either by exact match or by name prefix.
- **Tool registry and executor** — registers Go functions implementing each tool (`list_devices`, `get_device_state`, `control_device`) and dispatches them at runtime, with user identity carried via the request context.
- **Tool loop** — a bounded loop (currently 8 iterations max) that lets the LLM observe tool outputs and decide further actions before producing a final user-facing message.
- **Orchestrator** — the entry point that combines provider, tool loop, and tool definitions for a given chat turn, supporting both single-shot generation and SSE streaming.

6.2 Interaction flow

The typical conversational interaction flow is as follows:

1. A user logs in through `auth-web` and obtains a panel session (cookie `sid`).
2. The user opens the Chat view in `web`. The frontend lists existing chats and creates a new one if necessary.
3. The user sends a message such as "Turn on the LED strip and set it to warm white". The frontend issues `POST /chats/:id/messages?stream=true`.
4. `llm-service` persists the user turn, retrieves chat history, and assembles a tool-aware request to the provider resolved from the chat's model name.
5. The LLM invokes `list_devices` to discover available devices, then `control_device` with the inferred capability and value.
6. Each tool invocation runs inside the tool loop: the executor handles the call, writes the new state to PostgreSQL, and publishes an MQTT message to the device's command topic.
7. The target device (e.g., the WS2811 firmware) receives the MQTT command, executes it, and publishes the resulting state back to its state topic.
8. The model completes its reasoning, the final assistant message is streamed to the client via SSE, and the turn is persisted.
9. In parallel, if the user controls the device via Yandex Smart Home (voice), the request lands at the adapter and is translated into an MQTT message into the same broker — which means both paths converge on the same source of truth.

This architecture ensures loose coupling between components, supports horizontal scal-

ing, and allows flexible extension of both device types and integration targets.

7 Implementation Details

This section describes the implementation of the most important components beyond what is visible in the architecture diagram.

7.1 Authentication and authorization

`auth-service` is responsible for two distinct flows: user-facing authentication and serving as a first-party OAuth 2.0 provider for the panel SPA.

User-facing endpoints (`/auth/login`, `/auth/register`, `/auth/logout`, `/auth/me`) operate over an opaque session cookie (`session_id`) backed by the `oauth_sessions` table.

OAuth endpoints (`/oauth/authorize`, `/oauth/token`, `/oauth/refresh`, `/oauth/userinfo`, `/oauth/endsession`, `/oauth/jwks`) implement the authorization-code flow used by the panel SPA. JWTs are signed with RSA-4096 keys mounted into the container at `/app/keys`. The JWKS endpoint exposes the public key so that downstream services (`panel-api`, `devices-api`, `llm-service`) can validate tokens without sharing secrets.

The database table layout (extracted from `db/init/init.sql`) reflects this separation:

Листинг 1: Core authentication tables

```
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(256),
    email VARCHAR(256) NOT NULL UNIQUE,
    password TEXT NOT NULL,
    confirmed BOOL DEFAULT FALSE,
    created_at timestamptz NOT NULL DEFAULT now(),
    ...
);

CREATE TABLE oauth_sessions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    last_active timestamptz NOT NULL DEFAULT NOW(),
    created_at timestamptz NOT NULL DEFAULT NOW(),
    expires_at timestamptz DEFAULT NULL
);

CREATE TABLE panel_sessions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    token_type VARCHAR(25),
    access_token TEXT NOT NULL,
    refresh_token TEXT NOT NULL,
    ...
);
```

7.2 Device pairing flow

The pairing protocol uses a one-time code generated by the backend, transferred to the device through a captive portal, and confirmed by the device against the cloud API.

1. In `web`, the user clicks "Add device". The frontend calls `POST /devices/pairing/start`.
2. `devices-api` generates a 6-character code, stores the mapping *code* → *user_id* in Redis with TTL = 5 minutes, and returns `{code, expires_in}`.
3. The user powers on the unprovisioned ESP32. With no stored Wi-Fi credentials, the firmware enters AP mode and starts a captive portal at `192.168.4.1` (DNS captive enabled).
4. The user connects their phone to the AP `SmartHome-XXXXXX`, sees a styled captive form, and enters a human-readable device name, the home Wi-Fi SSID (with an in-page *Scan* button that calls `/scan` and offers a datalist of nearby networks), the Wi-Fi password, and the pairing code.
5. The firmware stores the credentials in NVS, switches to STA mode, connects to Wi-Fi, and calls `POST https://api.smarthome.hipahopa.ru/devices/pairing/confirm` with the code, device UID, MAC, and a device-type hint.
6. `devices-api` validates the code against Redis, persists the device under the corresponding user (with capability descriptors derived from the firmware's `describe` MQTT publish), and returns `{user_id, device_id}` to the firmware.
7. The firmware persists these IDs in NVS, connects to MQTT, subscribes to its command topics, and starts the runtime loop.

The firmware source is organized around a `CapabilityManager` that owns a list of `Capability` objects. Each capability exposes:

- `String describe()` — serializes the capability into JSON conforming to the Yandex Smart Home schema, published on the `.../describe` MQTT topic at startup;
- `bool handleSet(payload)` — handles a set-state command;
- `String state()` — serializes the current state for publication on `.../state`.

This abstraction is what allows the WS2811 firmware to add color and brightness capabilities by simply registering additional objects without touching the MQTT or pairing infrastructure.

7.3 LLM tool loop

The conversational layer is implemented in `llm-service`. The key abstraction is the provider interface:

Листинг 2: Provider interface

```
type Provider interface {
    Name() string
    Capabilities() Capability
    Generate(ctx context.Context, req *Request) (*Response, error)
```

```
    Stream(ctx context.Context, req *Request) (Stream, error)
}
```

Two concrete providers are implemented: an OpenAI-compatible client and an Anthropic Claude client (using the official `anthropic-sdk-go`). The provider router maps model names to providers:

- Exact match: `gpt-4o`, `gpt-4o-mini`, `gpt-5.2`, `o3`, `o4-mini` are routed to OpenAI;
- Prefix match: `claude-*` is routed to Anthropic (which covers `claude-opus-4-7`, `claude-sonnet-4-6`, etc., without hard-coding model lists).

The tool loop itself is intentionally simple, but sufficient for the agentic patterns we exercise:

Листинг 3: Core of the tool loop (simplified)

```
func (l *loop) Run(ctx context.Context, p clients.Provider, req *clients.
    Request) (*clients.Response, error) {
    for i := 0; i < l.maxIters; i++ {
        resp, err := p.Generate(ctx, req)
        if err != nil { return nil, err }
        if len(resp.ToolCalls) == 0 { return resp, nil }

        req.Messages = append(req.Messages, clients.Message{
            Role: clients.RoleAssistant, Content: resp.Content,
            ToolCalls: resp.ToolCalls,
        })
        for _, tc := range resp.ToolCalls {
            out, err := l.exec.Call(ctx, tc.Name, tc.Arguments)
            if err != nil { return nil, fmt.Errorf("tool %s failed: %w", tc.
                Name, err) }
            req.Messages = append(req.Messages, clients.Message{
                Role: clients.RoleTool, Name: tc.Name,
                ToolCallID: tc.ID, Content: out,
            })
        }
    }
    return nil, fmt.Errorf("tool loop: max iterations reached")
}
```

A maximum of eight iterations prevents accidental infinite loops while still allowing legitimate multi-step plans (e.g., list devices → query state of one → control it).

Three tools are registered:

- `list_devices` — returns the user's devices with IDs, names, rooms, and types;
- `get_device_state` — returns the full state of a specific device, including capabilities and sensor properties;
- `control_device` — sends a control command to a capability, after which the corresponding MQTT publish is made.

User identity is propagated to the tools through a typed context key, set by the orchestrator at the start of each chat turn:

```
ctx = context.WithValue(ctx, ctxkeys.UserID, req.UserID)
```

This means every tool call is automatically scoped to the calling user — a tool cannot accidentally read another user's devices even if the LLM produces an out-of-scope device ID.

7.4 Streaming responses

The chat endpoint accepts a `stream=true` query parameter. When set, `llm-service` uses the streaming variant of the tool loop: intermediate tool steps are still resolved with `Generate`, but the final non-tool-call response is streamed token-by-token to the client over SSE. The frontend renders these tokens incrementally in the chat view (`ChatMessages.vue`).

7.5 Devices API

In parallel with the Yandex adapter, the platform exposes a first-party REST API in `devices-api` that the panel SPA consumes directly. The Yandex adapter still owns the skill protocol; the Devices API serves the in-app experience.

- `GET /devices` — returns the user's devices with eagerly-loaded capabilities and properties.
- `GET /devices/:id` — returns a single device by id, scoped to the calling user.
- `PUT /devices/:id` — updates editable metadata (`name`, `room`, `description`). Partial-update semantics: only fields present in the body are touched.
- `DELETE /devices/:id` — soft-deletes the device through Gorm's `DeletedAt` column. The corresponding capabilities and properties cascade.
- `POST /devices/:id/capabilities/:type/set` — writes the desired state to PostgreSQL *and* publishes an MQTT message on `<user_id>/<device_id>/capabilities/<type>/set`. The firmware subscribes to this topic, executes the command, and publishes the resulting state back to the broker, which the listener then reflects into the database.

All endpoints are protected by the panel session cookie middleware (`sid`) and scope every query by `user_id`, so a tool cannot accidentally read or modify another user's devices even with a crafted ID.

The MQTT publishing happens in a dedicated `mqtt.Client` wired into the service container at startup with auto-reconnect and a 2-second publish timeout. If the broker is unreachable the API fails fast with `503` rather than silently dropping commands.

7.6 Yandex Smart Home adapter

The adapter implements the Yandex Smart Home skill API. Three core endpoints are exposed under `/user/devices` and protected by Yandex's bearer token plus the project's own JWT middleware:

- `GET /user/devices` — returns the user's devices in Yandex's schema, populated from PostgreSQL.
- `POST /user/devices/query` — returns the current state of the requested devices.
- `POST /user/devices/action` — accepts a list of state changes; the adapter publishes an MQTT command per device-capability pair and reports per-capability success based on the publish acknowledgment. The Yandex skill protocol does not require synchronous device confirmation; the eventual state is reflected back into PostgreSQL by the `StatusListenerService` (Section 7) as devices report on their `state` topics.

A background `StatusListenerService` subscribes to MQTT state topics for all paired devices and reflects updates back into PostgreSQL so that subsequent queries are served from a consistent local cache.

7.7 Frontend

The frontend is a single Vue 3 SPA (Vite 7, TypeScript) consuming the LLM service and the Devices API. It is organized around three screens served by the same shell.

HomeView. The landing screen. For unauthenticated visitors it renders the hero, brief value proposition, and a sign-in button. For authenticated users it personalizes the greeting and renders two interactive *overview* cards summarizing the user's state: a Devices card with three live counters (total, online, currently active) and a Chats card showing the number of conversation sessions. Both cards are click-through to the corresponding screens.

DevicesView. The main device management surface. It loads the user's devices via `GET /devices`, computes a hero summary (total/online/active), and offers a fuzzy search input over name, room, and type. Devices are grouped by room with an alphabetical ordering (devices without a room are placed last) and rendered as a responsive grid of `DeviceCard` components. Each card shows the device's icon (derived from `type`), an online dot, a per-capability quick toggle for on/off, and visual hints for brightness (a horizontal bar) and color (a colored dot) when the capability is present. Toggles use optimistic UI: the state is flipped locally first, the request is sent, and the change is reverted on error. Pairing is initiated through the existing floating action menu and `PairingModal`, both of which were retained from earlier checkpoints.

DeviceDetailModal. A dedicated modal opens on card click. It renders capability-aware controls dynamically from the device's capability list:

- `devices.capabilities.on_off` — a styled toggle.
- `devices.capabilities.range` — a horizontal slider that reads `min`/`max`/`precision` from the capability's `parameters` object.
- `devices.capabilities.color_setting` — a native HTML color picker plus a row of preset colors (warm white, cool white, indigo, red, green, amber).

- `devices.capabilities.mode` — a pill row of mode buttons rendered from the capability's `parameters.modes` array.
- `devices.capabilities.toggle` — a styled toggle per toggle-instance.

The modal also surfaces sensor readings from the device's `properties` array, an inline metadata editor (name, room, description), a meta information block (device ID, UID, last seen), and a two-step destructive button for unpairing the device.

ChatsView. The conversational surface introduced at Checkpoint 2, extended in the final iteration. It coordinates three components:

- `ChatSidebar` — lists chats, exposes new-chat, rename, and delete actions. Receives the active chat id as a prop so the highlight state is driven by the URL.
- `ChatMessages` — renders the message thread with Markdown (`marked`), shows tool-call boxes, and integrates with the streaming endpoint. Renders an empty-state placeholder (with clickable suggestion chips) both when no chat is selected and when a selected chat has no messages yet.
- `ChatInput` — composes new messages, triggers streaming, and exposes a `sendNow(text)` method used by the suggestion chips.

The active chat lives in the URL (`/chats/:chatId?`). On mount and on every URL change the view loads the corresponding history, so a browser reload restores the same chat instead of dropping the user on a blank screen. After a successful pairing in the modal the page is reloaded so that the newly paired device immediately appears in all relevant views (Devices list, Home overview, and the LLM `list_devices` tool results).

State sync. Device state in the Devices view is kept up to date through two channels: a fresh `GET /devices` on screen mount (which initialises the grid) and an SSE subscription to `GET /devices/stream` that reflects MQTT state events into the local Vue store in real time. This means that a state change triggered by the firmware, by Yandex Smart Home, or by another browser tab is visible in the panel within hundreds of milliseconds, without any polling.

7.8 Data model summary

The PostgreSQL schema reflects the major entities of the platform. Beyond the auth-related tables shown above, the following tables form the device and chat domains:

Листинг 4: Device and chat domain tables (excerpt)

```
CREATE TABLE manufactured_devices (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    secret TEXT NOT NULL,
    mac_address TEXT NOT NULL UNIQUE,
    registered BOOLEAN DEFAULT FALSE,
    ...
);
```

```
8
9  CREATE TABLE devices (
10     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
11     device_uid TEXT NOT NULL,
12     mac_address TEXT NOT NULL,
13     user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
14     name TEXT, description TEXT, room TEXT, type TEXT,
15     status_info JSONB, custom_data JSONB, device_info JSONB,
16     last_seen timestamptz DEFAULT NOW(),
17     ...
18 );
19
20 CREATE TABLE capabilities (
21     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
22     device_id UUID NOT NULL REFERENCES devices(id) ON DELETE CASCADE,
23     type TEXT NOT NULL CHECK (type IN (
24         'devices.capabilities.on_off',
25         'devices.capabilities.color_setting',
26         'devices.capabilities.mode',
27         'devices.capabilities.range',
28         'devices.capabilities.toggle')),
29     retrievable BOOL NOT NULL DEFAULT TRUE,
30     reportable  BOOL NOT NULL DEFAULT FALSE,
31     parameters JSONB, state JSONB
32 );
33
34 CREATE TABLE llm_chats (
35     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
36     user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
37     model VARCHAR(64) NOT NULL,
38     title TEXT NOT NULL DEFAULT '',
39     ...
40 );
41
42 CREATE TABLE llm_chat_message (
43     id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
44     chat_id UUID NOT NULL REFERENCES llm_chats(id) ON DELETE CASCADE,
45     role TEXT NOT NULL CHECK (role IN ('user','assistant','system','tool')),
46     model_name VARCHAR(64) NOT NULL,
47     input_tokens INTEGER DEFAULT 0,
48     output_tokens INTEGER DEFAULT 0,
49     content TEXT DEFAULT '',
50     tool_call_id VARCHAR(255), tool_name VARCHAR(255),
51     tool_args JSONB DEFAULT '{}'::jsonb,
52     tool_result JSONB DEFAULT '{}'::jsonb,
53     status llm_message_status NOT NULL DEFAULT 'completed',
54     ...
55 );
```

The capability data model deliberately follows the Yandex Smart Home convention (`devices.capabilities.*`, `devices.properties.*`) so that the adapter is a pass-through transformation rather than a translation layer.

7.9 Deployment

The system is deployed via Docker Compose. The compose file orchestrates the following services: `auth`, `panel-api`, `devices-api`, `llm-service`, `adapter`, `auth-web`, `web`, `nginx`, `db` (PostgreSQL 17), `redis` (Redis 8), and `broker` (Mosquitto 2.0.22). RSA keys for JWT signing are mounted from `./keys` as read-only volumes. The compose stack's internal `nginx` handles HTTP routing by hostname between the auth and panel frontends and the corresponding APIs; in the production deployment, a separate outer nginx instance with mounted Let's Encrypt certificates terminates TLS and forwards plaintext HTTP into the stack. This two-tier setup keeps TLS material and renewal logic outside the application compose project.

8 Functional Specification

This section enumerates the functionality delivered in the final submission.

8.1 Delivered functionality

Identity and access.

- Email/password registration and login (`auth-service`).
- OAuth 2.0 authorization-code flow with refresh tokens, end-session, JWKS endpoint.

Device pairing.

- One-time pairing code (5-minute TTL, Redis-backed).
- Device-side captive portal (DNS-captive, soft AP, NVS credential storage).
- Backend confirmation, capability discovery via the firmware `describe` publish.
- Edge cases handled: duplicate pairing attempts, network loss during pairing, code expiry.

Device control.

- MQTT pub/sub between firmware and backend.
- Capability/property data model aligned with Yandex Smart Home.
- REST API (`/devices` CRUD plus per-capability `set`) consumed by the panel UI.
- Server-Sent Events stream (`/devices/stream`) for real-time state updates with no polling.
- Panel UI: search, room grouping, capability-aware detail modal with on/off, brightness slider, color picker with presets, mode buttons, sensor readings, inline rename, and two-step delete.
- Conversational control: natural-language commands routed via LLM tool calls to MQTT.

Conversational AI.

- Multi-provider LLM router: OpenAI-compatible APIs and Anthropic Claude, selectable per-chat by model name.
- Persistent chat history per user with model and title metadata, automatic title generation from first turn.
- Tool definitions for device discovery, state inspection, and control.
- Bounded tool loop (max 8 iterations) with per-call user-scoped context.
- Streaming responses via Server-Sent Events.

Ecosystem integration.

- Yandex Smart Home skill: list, query, action endpoints; OAuth-based linking.
- MQTT state listener that reflects device state changes into PostgreSQL.

Reference firmware.

- `generic-firmware` — minimal ESP32 firmware with one on/off capability.
- `ws2811` — ESP32 firmware controlling a WS2811 LED strip with on/off, color, and brightness capabilities, built on top of `FastLED`.

8.2 Not yet delivered (acknowledged limitations)

- **Bring-your-own-key for LLM providers.** The `user_llm_keys` table is present in the schema and is intended to hold per-user, encrypted provider keys, but `llm-service` does not yet read it: the prototype uses a single deployment-wide API key from configuration. Wiring up the table is straightforward but outside the current scope.
- **Hardware-anchored device authentication.** The `manufactured_devices.secret` column is present for a future per-device challenge/response flow, but the current pairing protocol validates only the one-time code (Section 7).
- **MQTT broker hardening.** The broker currently runs with a single shared username/-password over plaintext TCP and is exposed on port 1883 for demo accessibility. Per-device credentials, TLS (port 8883), and topic ACLs are tracked in Appendix B as the immediate security backlog.
- **RAG / long-term memory.** Conversation context is currently limited to the active session. The vector store integration for cross-session memory retrieval is scaffolded (`rag.Retriever` interface) but not yet backed by a production embedding index.
- **Voice interface.** The smart-speaker integration is designed and partially scaffolded (channel enum, shared chat-service backend) but not yet functional — it requires a wake-word detector and an STT/TTS pipeline on the device.
- **Home Assistant bridge.** The adapter service includes the structural hooks for Home Assistant integration, but full bidirectional sync is pending.
- **Proactive automation.** Rules that trigger home actions based on inferred user context (e.g., end-of-workday detection) are not yet implemented.

9 Customer Development

To validate the pivot towards hyper-personalization and assess the value of a unified-context assistant, several semi-structured interviews were conducted with potential users from the target segment (technically literate individuals aged 22–35 with interest in smart home technologies).

9.1 Interview methodology

Each interview followed a problem-first approach [24]: participants were first asked about their current use of smart home and productivity tools, then introduced to the hyper-personalization concept, and finally invited to react to specific usage scenarios. Interviews lasted approximately 20–30 minutes and were conducted online or in person.

9.2 Interview insights

Overall, participants described managing multiple disconnected tools for work and home. Those already using smart home systems mentioned switching between several apps and lacking integration between their calendar, task manager, and home automation. The idea of a single assistant aware of both work context and home environment was perceived as intuitive and appealing.

Participants who did not use smart home devices were still interested in the chat-based assistant as a productivity tool. For them, the main value was conversational quality and the feeling that the assistant "understands" their day. This suggests that the chat interface itself can deliver value even without hardware integration — a useful observation for go-to-market sequencing.

Privacy concerns were raised by more technically experienced users, particularly regarding cloud-based language models and work-related data. Local model support (Ollama) and flexible architecture were seen as important trust-building factors.

Voice interaction was viewed as useful but only under strict performance expectations. Participants indicated that noticeable latency would significantly reduce usability compared to simple manual controls.

Several interviewees also expressed interest in proactive behavior. Instead of issuing explicit commands, they would prefer the assistant to infer context (e.g., end of workday, long meetings) and adjust the environment or provide summaries accordingly.

9.3 Key findings

1. Users experience fragmentation between productivity tools and smart home systems as a real friction point.
2. Privacy and the option of local processing matter to technically literate users.
3. Low latency is critical for voice-based interactions but only "nice to have" for chat.

4. The chat interface has standalone value as a productivity assistant.
5. There is demand for context-aware and proactive assistant behavior beyond pure command execution.

These findings directly shaped the final feature set: per-user API keys to support a future local-model pathway, tool-use over hand-crafted intent parsing to keep the conversational quality high, and explicit context propagation in the tool layer to enable proactive logic in future iterations.

10 Experiments and Results

This section reports the qualitative and small-sample quantitative evaluation of the prototype. The figures below come from *spot measurements* harvested from production access logs of the live deployment at `smarthome.hipahopa.ru` and from the ESP32 serial monitor during integration testing on 2026-05-27. Sample sizes are intentionally modest — this is a prototype evaluation, not a production benchmark, and honest small-n numbers are preferred over fabricated large-n tables. Every number in this section can be traced back to a specific log line or screen capture in the project artefacts.

10.1 End-to-end demonstration

The most important result is qualitative: the full end-to-end scenario works on real hardware.

- A factory-flashed ESP32 was powered on, joined the local AP `SmartHome-XXXXXX`, accepted SSID, password, device name, and pairing code through the captive portal, rebooted, joined the home Wi-Fi, exchanged the pairing code with `devices-api`, and appeared in the Devices view of `smarthome.hipahopa.ru` within seconds. The device's capability descriptor (on/off, brightness, color) was registered through the firmware's MQTT `describe` message and exposed in both the panel UI and the LLM `list_devices` tool.
- Natural-language commands issued in the chat ("turn on the LED strip", "set it to warm white", "turn it off") were translated by the LLM into `control_device` tool calls; the resulting MQTT `.../set` publishes were received by the device (visible in its serial log as `[MQTT] RX ... set payload=...`), routed to the corresponding capability, and reflected as an immediate physical change on the strip — e.g. a smooth brightness ramp from the current value to the requested target.
- The same device was also controlled from the panel UI through `POST /devices/:id/capabilities/:`. Both paths converge on the same MQTT topic, so a chat-initiated change immediately propagates back into the panel's live state stream and vice versa.

The pairing flow was iterated and validated informally during development; no formal large-sample reliability study was run for the final submission. Several real failure modes were surfaced and fixed during integration — query-string stripping in the inner nginx (cascade-affecting both the pairing-status poll and the chat `?stream=true` flag) and a request-context cancellation that prevented the assistant message from being persisted after streaming. Both fixes are visible in the project's git history.

10.2 Worked example: chat-driven LED strip control

This subsection traces a concrete chat session from the production deployment on 2026-05-27 (chat id `e0e3017b-9b33-4669-aae9-3cf27ff71e0d`, model `gpt-4o`). The transcript was reconstructed from the `llm_chat_message` table in PostgreSQL and from the corresponding adapter and ESP32 logs — nothing in this subsection is fabricated; every line below has a

corresponding production-log artefact.

Step 1 — "what devices do I have?" **User** (Russian): *"привет, какие у меня есть устройства?"*

The LLM issued the tool call `list_devices()`. The tool returned a single device with id `c135e79e-af77-4e54-86d6-78d4291583c8`, name `esp32-DC84D015B80C`, type `devices.types.light.s` and three capabilities (`on_off`, `range/brightness`, `color_setting`).

Assistant: *"Привет! У вас есть одно устройство:* ***Имя:*** *esp32-DC84D015B80C,* ***Тип:*** *Светодиодная лента (light strip)..."*

Total wall-clock for this turn: **2.03 s** (Table 4, row 1).

Step 2 — "turn it on" **User:** *"можешь включить?"*

The LLM issued the tool call `control_device(device="esp32-DC84D015B80C", capability="c` The handler resolved the device by name to its UUID (recall the previously fixed bug where the model passed the human-readable name instead of the UUID), persisted the desired state via `POST .../capabilities/devices.capabilities.on_off/set`, and published the corresponding MQTT message:

```
topic:  c2701344-5365-4bd4-adbf-0588041bf030/
        c135e79e-af77-4e54-86d6-78d4291583c8/
        capabilities/devices.capabilities.on_off/set
payload: {"state":{"instance":"on","value":true}}
```

The ESP32 serial log shows the matching reception: `[CAPMAN] routed to devices.capabilities.on_` followed by a smooth brightness ramp `[WS_BRIGHTNESS] current: 0 -> 133`.

Assistant: *"Светодиодная лента включена."*

Total wall-clock: **5.05 s** (Table 4, row 6 — single-tool turn).

Step 3 — "and now turn it off" **User:** *"а теперь выключи"*

Analogous `control_device` call with `value=false`. The adapter log records the same topic; the firmware reports `[CAPMAN] routed to devices.capabilities.on_off, ok=1`; the brightness ramps `133 -> 0`.

Assistant: *"Светодиодная лента выключена."*

Step 4 — introspecting the device **User:** *"а что ты можешь с ней ещё сделать?"*

This turn used no tool calls; the model relied on the capabilities it had already seen in step 1 (the conversation context is retained server-side). It summarised that the strip supports on/off, brightness, and colour setting. This is a useful negative example: the system does not waste a tool round-trip when the answer is already in the context.

What this example demonstrates. The session exercises the four interlocking pieces of the system: (i) the LLM tool router (`list_devices`, `control_device`); (ii) the persistence path `POST /devices/.../set`; (iii) the MQTT bus with the user-id/device-id/capability topic layout (Appendix D); (iv) the firmware capability dispatcher in the ESP32. The same physical strip is then also controlled from the panel UI (`DevicesView`) by clicking the on/off toggle; the resulting MQTT publish is identical, confirming that chat and panel are two interchangeable surfaces over the same underlying device-state contract.

10.3 Server-side latency for a device command

The cleanest single-component measurement is the server-side handler time of `POST /devices/:id/capa` which (a) authenticates the request against the panel session, (b) writes the desired state to PostgreSQL, and (c) publishes the MQTT `.../set` message. Three consecutive commands captured in the production `devices-api` log:

Таблица 3: Server-side latency of capability `set` (production, 2026-05-27, $n = 3$)

Sample	**Wall-clock**
`range/set` (brightness ← 54)	2.70 ms
`on_off/set` (off)	2.36 ms
`on_off/set` (on)	2.33 ms

These numbers represent only the path through `devices-api` — they exclude Wi-Fi transit to the device and the firmware's own scheduling. From the ESP32 serial monitor it is visible that all three commands arrived at the device and were routed to their handlers (`[CAPMAN] routed to devices.capabilities.on_off, ok=1` for the on and off events, plus the brightness ramp updating the FastLED-driven strip). The wire-level RTT was not directly instrumented for the final report; on the development Wi-Fi the perceived end-to-end delay was sub-second.

10.4 Chat response duration (total wall-clock)

The chat endpoint streams its response over Server-Sent Events, so a meaningful single number would be the time-to-first-token rather than total request duration. The current instrumentation only records total wall-clock per request, which conflates network, the LLM call, and (for tool-driven turns) any tool-loop `Generate` rounds. With that caveat, the following are all the chat `POST /chats/:id/messages?stream=true` requests captured in the production `llm-service` log during integration testing on 2026-05-27 (model: `gpt-4o`):

The distribution clearly stratifies: requests answered directly by the LLM land in the 2–3.5 s range, requests that involve one tool call add roughly another 1–2 s for the extra `Generate` round inside the tool loop, and a multi-step plan (here: list devices, then control, then read state to confirm) lands around 10 s. The dominant component throughout is the cloud LLM

Таблица 4: Total wall-clock per chat stream request (production, 2026-05-27, $n = 7$)

Request	**Wall-clock**
Plain Q&A (introductory message)	2.03 s
Tool-free reply	2.54 s
Tool-free reply	2.96 s
Tool-free reply	3.29 s
Plain reply	3.47 s
With one `control_device` call	5.05 s
With multi-step tool plan (list + control + state)	10.18 s

call — backend overhead from logs is sub-10 ms per request.

The Checkpoint 2 target of < 500 ms for an intent-routed device action was unrealistic with a cloud LLM in the loop; it referred to a hypothetical configuration in which intent parsing would be done by a local Ollama model or by a side-channel classifier. Under the actual cloud configuration shipped in the final prototype, even the fastest single-tool turn cannot meet that target. This is an honest finding and is reflected in the conclusions.

10.5 Reconnection resilience

The firmware reconnect path was exercised during development. Forcing a router reboot while a device was paired and online caused the firmware to lose its MQTT session, retry, and reattach within several seconds; on reattach it re-subscribes its `.../set` topics and republishes `describe` and a full state snapshot, so no manual intervention is required. The reconnect interval was not formally instrumented for the report; the observed wait was on the order of ~5 s.

10.6 Scalability (conceptual evaluation)

Each microservice in the system is stateless with the exception of persistent state, which is delegated to PostgreSQL, Redis, and the MQTT broker. The chat tool loop is per-request and does not maintain in-process global state. A synthetic load study was *not* run for the final submission; based on the architecture, the bottleneck under sustained chat load is expected to be the upstream LLM provider (rate-limited) rather than any of the Go services, all of which use connection pooling and stateless request handlers. A proper load study with response-time and CPU curves is left as future work.

11 Roadmap, Timeline, and Execution

11.1 Iterative development tracks

Development followed iterative and parallel engineering tracks, with integration milestones between them:

- **Backend track:** core services, APIs, and event-driven pipelines (completed).
- **Device track:** ESP32 firmware framework for pairing, capability publication, and reconnection (completed for two reference devices).
- **Frontend track:** web interface for onboarding and device control (completed); chat interface added at Checkpoint 2 (completed).
- **AI/Chat track:** Chat service with LLM integration, tool calls, and streaming (completed; RAG scaffolded).
- **Integration milestones:** backend + firmware tested on local broker (passed); frontend + backend integration verified with test devices (passed); bridges to Yandex Smart Home validated (passed); Home Assistant bridge stubbed.

11.2 Project timeline

The development of the Hipahome platform followed a phased, sequential approach with concurrent device-engineering work. The four stages are summarized below.

1. **Stage 1: OAuth and identity (October–November 2025)** — foundational user authentication and API access, including the OAuth provider and JWT issuance.
2. **Stage 2: Yandex Smart Home adapter (November–December 2025)** — integration with the external smart home ecosystem; MQTT topic and message-format design.
3. **Stage 3: Generic firmware and pairing (December 2025 – February 2026)** — device onboarding and unified firmware architecture: captive portal, MQTT runtime, capability publication.
4. **Stage 4: Chat service and LLM integration (February–April 2026)** — the pivot introduced at Checkpoint 2: a new microservice for chat sessions, multi-provider LLM router, tool loop, streaming responses.
5. **Stage 5 (additional, post-CP2): Device engineering and reference WS2811 firmware (April–May 2026)** — assembly of the second reference device with color and brightness capabilities, demonstrating the capability framework.

11.3 Deviations from the initial plan

The most substantial deviation from the Checkpoint 1 plan was the Checkpoint 2 pivot: the project's scope was expanded to include a conversational assistant layer that had not been part of the original specification. This pivot was directly motivated by the customer development findings discussed in Section 9 and was executed without compromising the original middleware

Рис. 2: Gantt chart: Hipahome project timeline (October 2025 – May 2026, abstract weeks).

deliverables.

A second deviation, smaller in scope, was the decision to add a second reference firmware (WS2811) in Stage 5. The original plan called for only the generic ESP32 firmware as a reference; the WS2811 firmware was added because the capability framework needed a multi-capability device to be validated meaningfully.

12 Lean Canvas

The Lean Canvas was revised to reflect the pivot towards hyper-personalization and the expanded product scope [25].

13 Conclusion

This project delivers a working prototype of Hipahome, a smart home software platform that combines flexible integration of user-defined devices with a unified, context-aware conversational assistant. The final submission demonstrates an end-to-end scenario: a new user registers, pairs a physical ESP32 LED strip through a captive portal, and controls it from a web chat through natural-language commands that are routed through a multi-provider LLM tool loop and an MQTT message bus to the device.

13.1 Summary of contributions

Concretely, the project contributes:

- A six-service Go backend (`auth-service`, `panel-api`, `devices-api`, `llm-service`, `adapter`, `registry`) with clear ownership boundaries and shared infrastructure.
- A captive-portal-based pairing flow built on a one-time pairing code, with the database schema prepared for a future per-device cryptographic binding. The flow was validated end-to-end on a physical ESP32 WS2811 strip and was extended in the final iteration to also capture a human-readable device name.
- A multi-provider LLM service that abstracts over OpenAI-compatible APIs and Anthropic Claude, supports structured tool calls, streams responses over SSE, and propagates user identity to tools through context.
- A pluggable capability framework on ESP32 demonstrated with two reference firmwares (generic on/off and WS2811 with color/brightness).
- A working Yandex Smart Home adapter that translates between Yandex's skill protocol and the platform's internal MQTT bus.
- A server-side device-command path that handles `POST .../set` in ∼2–3 ms (observed in production), with the wire-level RTT to the device dominated by Wi-Fi latency.
- A streamed chat path delivering tool-free turns in ∼2–3.5 s of total wall-clock time on `gpt-4o`, increasing as the tool plan grows (Section 10).

13.2 Observed trade-offs and honest limitations

The most important honest finding is that the *end-to-end* latency of a conversational device action (Section 10) is dominated by the cloud LLM round-trip. Tool-free turns currently land in 2–3.5 s, single-tool turns add roughly another 1–2 s, and multi-step tool plans can run to 10 s. The Checkpoint 2 target of < 500 ms was unrealistic in the cloud-only configuration that the final prototype actually uses; reaching it would require a locally-hosted model (Ollama) or a side-channel intent classifier. This is left as future work.

A second limitation is the device-identity story: the pairing flow currently validates only the one-time code. The schema is ready for a per-device challenge/response (`manufactured_devices.se` but wiring this into `pairing_service.ConfirmPairing` is part of the security backlog, not part

of the delivered prototype. The same applies to per-user LLM keys (`user_llm_keys` table) and to MQTT broker hardening, both detailed in Appendix B.

A third limitation is the RAG layer. The architectural seams are in place (a `rag.Retriever` interface and a corresponding augmenter), but the production vector index is not yet wired up, so cross-session memory does not yet work. This is the next milestone for the conversational track.

Finally, the voice interface is designed and structurally scaffolded (channel enum, common chat backend), but the on-device wake-word and STT/TTS pipeline has not been built. This is intentional: the customer-development findings (Section 9) showed that voice is valuable but extremely latency-sensitive, and the current end-to-end latency does not yet meet the bar that voice interaction would require.

13.3 Future work

Three concrete directions are planned:

1. **RAG-backed cross-session memory.** Implement a per-user vector store (initially backed by pgvector for operational simplicity), populate it from completed chat turns, and integrate the existing retriever interface into the orchestrator.
2. **Local model path.** Add an Ollama provider to the LLM router so that latency-sensitive scenarios (especially voice) can run end-to-end on local hardware, addressing both performance and privacy concerns surfaced in interviews.
3. **Home Assistant bridge.** Extend the adapter service with a bidirectional Home Assistant integration so that devices paired in Hipahome appear natively in HA dashboards and HA-managed devices can be controlled from the Hipahome chat.
4. **Proactive automation.** Build a rule engine on top of the chat history that can trigger home actions based on inferred user context (e.g., end-of-workday detection, calendar-based modes), addressing the interview finding that users want a contextual companion, not just a command executor.

13.4 Closing remarks

Hipahome started as a smart home middleware and grew into a conversational assistant platform that uses the smart home as one of its surfaces. The pivot was deliberate, was grounded in user interviews, and is reflected in both the architecture (a dedicated AI layer) and the final user experience (chat as a first-class surface alongside device control). The result is a prototype that demonstrates the technical feasibility of the unified-context vision and provides a solid foundation for the future work outlined above.

References

[1] OASIS. *MQTT Version 5.0 Specification*. URL: https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html (accessed: 12.01.2026).

[2] Paulus Schoutsen. *Home Assistant — Open Source Home Automation*. URL: https://www.home-assistant.io/ (accessed: 12.01.2026).

[3] Yandex. *Smart Home API Documentation*. URL: https://yandex.ru/dev/dialogs/smart-home/doc/ru/ (accessed: 12.01.2026).

[4] Espressif Systems. *Arduino core for the ESP32*. URL: https://github.com/espressif/arduino-esp32 (accessed: 12.01.2026).

[5] Arduino. *Arduino Official Documentation*. URL: https://www.arduino.cc/en/Reference/HomePage (accessed: 12.01.2026).

[6] Docker, Inc. *Docker Compose Documentation*. URL: https://docs.docker.com/compose/ (accessed: 12.01.2026).

[7] Docker, Inc. *Docker Compose File Reference*. URL: https://docs.docker.com/compose/compose-file/ (accessed: 12.01.2026).

[8] PostgreSQL Global Development Group. *PostgreSQL Documentation*. URL: https://www.postgresql.org/docs/ (accessed: 12.01.2026).

[9] Redis Ltd. *Redis Documentation*. URL: https://redis.io/docs/latest/ (accessed: 12.01.2026).

[10] Eclipse Foundation. *Eclipse Mosquitto MQTT Broker*. URL: https://mosquitto.org/ (accessed: 12.01.2026).

[11] Eclipse Foundation. *Eclipse Mosquitto Docker Image Reference*. URL: https://hub.docker.com/_/eclipse-mosquitto (accessed: 12.01.2026).

[12] NGINX. *NGINX Docker Compose Integration Samples*. URL: https://docs.docker.com/samples/nginx/ (accessed: 12.01.2026).

[13] Gin Gonic. *Gin Web Framework Documentation*. URL: https://gin-gonic.com/docs/ (accessed: 12.01.2026).

[14] Vue.js. *Vue.js Official Documentation*. URL: https://vuejs.org/guide/introduction.html (accessed: 12.01.2026).

[15] OpenAI. *OpenAI API Reference*. URL: https://platform.openai.com/docs/api-reference (accessed: 01.03.2026).

[16] Anthropic. *Anthropic Messages API Reference*. URL: https://docs.anthropic.com/en/api/messages (accessed: 01.05.2026).

[17] Lewis, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv:2005.11401. URL: https://arxiv.org/abs/2005.11401 (accessed: 01.03.2026).

[18] Ollama. *Ollama — Run Large Language Models Locally*. URL: https://ollama.com/ (accessed: 01.03.2026).

[19] Mozilla. *Server-Sent Events API*. MDN Web Docs. URL: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events (accessed: 01.03.2026).

[20] Anthropic. *anthropic-sdk-go*. GitHub. URL: https://github.com/anthropics/anthropic-sdk-go (accessed: 01.05.2026).

[21] Daniel Garcia et al. *FastLED Library for Arduino*. URL: https://fastled.io/ (accessed: 01.04.2026).

[22] The Gorm Authors. *Gorm — The Fantastic ORM Library for Golang*. URL: https://gorm.io/ (accessed: 01.04.2026).

[23] PlatformIO Labs. *PlatformIO Documentation*. URL: https://docs.platformio.org/ (accessed: 01.04.2026).

[24] Blank, S. *The Four Steps to the Epiphany*. K&S Ranch, 2013.

[25] Maurya, A. *Running Lean: Iterate from Plan A to a Plan That Works*. O'Reilly Media, 2012.

Appendix A — Docker Compose

Листинг 5: Abbreviated docker-compose.yml

```yaml
services:
  auth:
    build:
      context: ./auth-service
    ...
  adapter:
    build:
      context: ./adapter
    ...
  panel-api:
    build:
      context: ./panel-api
    ...
  devices-api:
    build:
      context: ./devices-api
    ...
  llm-service:
    build:
      context: ./llm-service
    ...
  auth-web:
    build:
      context: ./auth-web
    restart: unless-stopped
  web:
    build:
      context: ./web
    restart: unless-stopped
  nginx:
    image: nginx:1.29
    ...
  db:
    image: postgres:17
    ...
  redis:
    image: redis:8.0
    ...
  broker:
    image: eclipse-mosquitto:2.0.22
    ...
```

Appendix B — Notes on security and device identity

This appendix documents the current security posture of the prototype honestly, distinguishing between what is implemented and what is scaffolded.

Identity claims at pairing time. The device transmits its hardware MAC and a stable UID along with the one-time pairing code. The `devices-api` validates the code against Redis but does *not* currently challenge the device on a per-device secret. The `manufactured_devices` table reserves a `secret` column for a future challenge/response binding, and the repository exposes `FindManufacturedByMAC` for that purpose; wiring this into `pairing_service.ConfirmPairing` is the planned next step. As a result, in the current prototype any device that obtains a fresh code from a logged-in user can claim ownership; protection against an attacker on the same Wi-Fi rests on the short code TTL (5 minutes) and on the code being delivered out-of-band.

Service-to-service. Authentication uses RSA-4096-signed JWTs whose public keys are available via the JWKS endpoint exposed by `auth-service`. The panel SPA obtains an access token through the OAuth authorization-code flow; the token is short-lived (15 minutes), refreshable, and revocable.

TLS termination. The compose stack speaks plaintext HTTP internally. In the production deployment, an outer nginx instance, configured outside this compose project, terminates TLS using a Let's Encrypt wildcard certificate and forwards into the stack.

MQTT broker. The broker is Eclipse Mosquitto 2.0.22 with a single `username/password` credential pair shared between backend services and devices, configured through a checked-in `mosquitto/config/passwd` (the password is a deployment placeholder that ships with the repository). For the demo deployment, the broker is currently exposed on port 1883 in plaintext so that physical ESP32 devices in the lab can reach it. This is explicitly insecure and is documented as the most important security debt. The immediate backlog, in order:

1. Rotate the shared password to a deployment-generated value, store in `.env`, regenerate `passwd`.
2. Add topic ACLs to mosquitto so a leaked device credential cannot read or publish on another user's topics.
3. Generate per-device MQTT credentials at pairing time, anchored to the `manufactured_devices.sec` column.
4. Migrate to MQTT over TLS (port 8883) using the same certificate chain as the HTTPS endpoints and adapt the firmware to `WiFiClientSecure`.
5. Close port 1883 on the public interface after the TLS migration.

LLM provider keys. The current prototype dispatches LLM requests using a single API key sourced from environment configuration. The schema includes a `user_llm_keys` table

reserved for per-user, encrypted-at-rest keys ("bring-your-own-key"), but `llm-service` does not yet consume it.

Appendix C — API endpoints (abridged)

Appendix D — MQTT topic layout

All device topics are rooted at `<user_id>/<device_id>/`. The following sub-topics are used:

The `describe` message is retained, which means a freshly-subscribed adapter or LLM tool immediately receives the device's capability list without having to wait for the next state publication. The `status` topic is currently also published as a retained "online" marker rather than a true MQTT Last-Will-and-Testament; switching to LWT is on the security backlog.

Таблица 5: Final Lean Canvas

Problem	(1) Smart home assistants and productivity AI tools are siloed — they share no context. (2) Voice assistants react to commands but do not understand the user's day. (3) Custom device integration remains technically complex (YAML, scripting).
Customer segments	Primary: technically literate users (developers, designers, researchers, 22–35) who use both smart home devices and AI productivity tools. Secondary: makers and hardware enthusiasts building custom devices.
Unique value proposition	One assistant. One context. Knows your work, knows your home. Control devices and solve tasks through natural conversation — the same session, the same memory, across screen and (in future) voice.
Solution	Unified chat microservice (Go) with persistent session memory and pluggable LLM backend (OpenAI-compatible APIs, Anthropic Claude; planned: Ollama for local). MQTT-based home device control triggered by conversational intents through structured tool calls. Hardware-bound pairing for custom ESP32 devices. Adapter bridges to Home Assistant and Yandex Smart Home.
Channels	GitHub (open source), technical communities (Habr, Reddit r/homeassistant), university demos, word-of-mouth among maker communities.
Revenue streams	Research prototype (no monetization at this stage). Future: SaaS subscription for hosted context + LLM; hardware kits for ESP32 devices; consulting for enterprise deployments.
Cost structure	Developer time (primary). LLM API costs (OpenAI / Anthropic; the schema is prepared for a future bring-your-own-key model that would shift the cost to end-users). Cloud hosting for prototype demo. Hardware components for device prototypes.
Key metrics	Server-side device command latency (`/devices/.../set`, observed: ∼2–3 ms in production). Chat response total wall-clock per stream (observed: ∼2–3.5 s for tool-free turns, longer when the LLM plans multiple tool calls). Pairing: end-to-end demonstrated on a physical WS2811 device (no formal reliability study).
Unfair advantage	Unified firmware + one-time-code pairing + adapter model for automatic ecosystem exposure, combined with an integrated conversational layer that grounds free-form interaction in physical state. No competitor in the consumer smart-home space combines both surfaces in a single open prototype.

Service	Endpoint	Auth
auth-service	`POST /auth/register`	—
auth-service	`POST /auth/login`	—
auth-service	`GET /auth/me`	session cookie
auth-service	`POST /oauth/authorize`	session cookie
auth-service	`POST /oauth/token`	client
auth-service	`POST /oauth/refresh`	refresh
auth-service	`GET /oauth/userinfo`	access JWT
auth-service	`GET /oauth/jwks`	—
devices-api	`POST /devices/pairing/start`	session
devices-api	`POST /devices/pairing/status`	session
devices-api	`POST /devices/pairing/confirm`	device key
devices-api	`GET /devices`	session
devices-api	`GET /devices/:id`	session
devices-api	`PUT /devices/:id`	session
devices-api	`DELETE /devices/:id`	session
devices-api	`POST /devices/:id/capabilities/:type/set`	session
devices-api	`GET /devices/stream (SSE)`	session
llm-service	`GET /chats`	session
llm-service	`POST /chats`	session
llm-service	`GET /chats/:id`	session
llm-service	`PUT /chats/:id`	session
llm-service	`DELETE /chats/:id`	session
llm-service	`POST /chats/:id/generate-title`	session
llm-service	`GET /chats/:id/messages`	session
llm-service	`POST /chats/:id/messages[?stream=true]`	session
adapter	`GET /user/devices`	Yandex bearer
adapter	`POST /user/devices/query`	Yandex bearer
adapter	`POST /user/devices/action`	Yandex bearer
adapter	`POST /user/unlink`	Yandex bearer

Таблица 6: Public HTTP endpoints by service.

Sub-topic	Direction	Payload
`describe`	device → broker (retained)	JSON capability descriptor (Yandex-style)
`state`	device → broker	JSON heartbeat / state snapshot
`capabilities/<type>/state`	device → broker	JSON state for one capability
`capabilities/<type>/set`	broker → device	JSON command `{instance, value, ts}`
`status`	device → broker (retained)	`{"status":"online"}`

Таблица 7: MQTT topic layout.